Bounded-Memory Scoped Symbol Tables on Compiler-Generated Traces: What Allocation-Free Storage Buys and What Interning Adds

Shubhi Singh and Adhyan Jain
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, India

***Abstract*—A compiler that must run in little memory needs a symbol table whose size is bounded in advance and whose behaviour at the bound is defined. We study how much of that goal is reached by storage discipline alone and how much by the table's design. Three tables are compared in depth: METIS-X, our earlier Robin Hood table with one slot per declaration; a conventional chained table over an entry arena, written here as an allocation-free baseline; and STRATA, which interns each visible name once, keeps the innermost declaration in the name's record and saves hidden declarations on an undo stack. Both new tables live in one caller-suppliable buffer and refuse an operation without changing state when it is full. All tables replay the symbol-table calls that an instrumented Tiny C Compiler makes on 8 open-source projects, with 31 interleaved rounds and paired statistics. The conventional table already needs 55–77% less memory than METIS-X and is 11–36% faster. STRATA needs a further 16–18% less memory than the conventional table, 27–36 bytes per live declaration, but is slower than it on 7 of 8 traces (median 29%); it is faster on synthetic workloads with heavy shadowing. Most of the improvement over METIS-X therefore comes from removing per-name allocation, not from STRATA's specific structure. We give a correctness argument for the all-or-nothing behaviour with tests that include adversarial collisions, and we run STRATA on an emulated bare-metal Cortex-M3 with 64 KiB of RAM. Nothing was integrated into a compiler or run on hardware, and we claim no new algorithm.**

***Index Terms*—symbol table, lexical scope, embedded compilers, memory budget, arena allocation, trace replay**

I. Introduction

A scoped symbol table maps identifiers to declarations and forgets a block's declarations when the block ends [1]. For a compiler that runs on a small device, or inside a tool with a fixed memory budget, three properties matter that general-purpose containers do not give: a memory bound fixed before compilation starts, no allocator calls during compilation, and a defined result when the bound is reached.

This paper continues the METIS-X project [2], a flat Robin Hood table [3] with one 32-byte slot per declaration and short names stored in the slot. Its authors closed it with three findings: every component was known, the traces were extracted from source text by regular expressions, and a string-keyed table has no place in a compiler that interns identifiers first, as the Tiny C Compiler (TCC) [4] does.

A first version of this work reported that a redesigned table, STRATA, used far less memory than METIS-X. A review of that version asked whether the gain came from the design or simply from no longer allocating per name. This paper answers that question, and its claims are limited accordingly:

1) An audit of METIS-X with one reproduced correctness defect (Section II).
2) Two bounded-memory tables with the same storage discipline: a conventional arena table and STRATA (Section III), with a correctness argument and tests for refusal without state change (Section IV).
3) Traces of the real symbol-table calls of TCC on 8 projects (Section V).
4) A decomposition of the improvement over METIS-X into the part due to allocation-free storage and the part due to STRATA's structure, with paired statistics (Section VI).

We do not claim a new algorithm; Section VII names the source of each mechanism.

II. Audit of the Previous Implementation

We read METIS-X at commit `7b3dacf`. *Hard failure under deep shadowing:* all declarations of one name share a hash value and one probe cluster; the probe distance is an 8-bit field capped at 250, and an insertion beyond it is refused. On a trace that declares one name in 400 nested scopes METIS-X refused 149 declarations and returned 298 wrong lookup results; no other table failed. *Lookup cost grows with shadowing:* every live declaration of the name is compared. *Allocation per long name:* names over 12 bytes are heap blocks. By inspection only: rehash is not exception-safe, the slot array never shrinks, each scope owns a heap vector. In the earlier evaluation the heap figures included harness data, the main baseline never reclaimed names, and latency was timed per call with a clock as expensive as the call.

III. Two Bounded-Memory Tables

Both tables keep all state in one array of 32-bit words that the caller may supply (fixed mode) or that grows by half through `realloc` (growable mode). Neither throws; in fixed mode neither allocates.

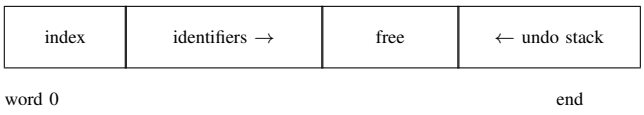


Fig. 1. STRATA's buffer. Identifier references are relative to the start of their region and undo references to the end of the buffer, so either boundary can move without patching stored references.

A. Conventional arena table

This is the classic compiler design, as in lcc-style compilers and TCC's symbol stacks: bucket heads at the bottom of the buffer, a stack of entries above them, scope marks at the top. An entry (16 bytes plus the unpacked name) is created for every declaration and linked at the head of its hash chain, so the first match in a chain is the innermost declaration. Scope exit unlinks and pops the scope's entries.

B. STRATA

The index is an open-addressing array with linear probing; a slot holds an 8-bit fingerprint and a 24-bit reference. The identifier stack holds one record per distinct visible name: value, a meta word (name length in words, type tag, scope depth of the innermost declaration) and the name, packed five characters to a word in the manner of RADIX-50 [5]. The undo stack grows down from the end of the buffer (Fig. 1).

A lookup packs and hashes the query, probes, and reads the innermost declaration from the record; its cost does not depend on shadowing depth. A declaration of a new name appends a record, fills a slot and pushes that slot's index (one word) on the undo stack. A declaration that shadows pushes the record's current value and tag (three words) and overwrites them. A redeclaration at the current depth replaces the value. Scope exit pops entries down to the scope's mark: a shadow entry restores what it saved; a first-declaration entry empties the recorded slot. Because the index grows by re-inserting records oldest first, the entries of first declarations are rewritten after each rebuild.

Limits: names of 1–255 bytes, depth 16,383, buffer 64 MiB. A rebuild is linear in the number of live names. The hash, MurmurHash3's 32-bit mixer [6], is unkeyed.

IV. Correctness of the Guarantees

Scope exit needs no tombstones. In linear probing without displacement a key is stored in the first empty slot of its probe path and never moves, so the path of a key consists of slots filled by older keys. The name being removed at scope exit is the newest key in the index; it lies on no remaining key's path, and emptying its slot cuts no path. The same holds for each next entry. Rebuilding oldest first preserves the premise.

All-or-nothing. Each operation validates, reserves, then writes. The reservation step either guarantees room for every word and index slot the operation will write, or fails before anything is modified; in growable mode the only fallible step, the allocator call, comes first. After it, no step can fail.

TABLE I
Compiler traces

Trace	Units	Events	Declare	Lookup	Names	Peak live	Depth
cjson	2	20,874	5,247	9,194	1,366	1,067	10
freertos	9	37,875	10,346	17,417	2,226	1,505	16
littlefs	2	21,080	3,835	12,230	1,001	732	16
lua	34	188,509	58,619	86,887	3,622	1,365	14
mbedtls	105	983,008	319,886	438,876	7,622	2,302	14
sqlite	1	268,960	27,072	172,426	7,801	4,245	22
tinycc	1	89,135	13,776	54,623	6,704	5,509	27
zlib	15	58,058	16,341	27,405	1,041	571	23

Tests. A digest of the complete state (counters, index, both stacks) is taken before every operation and must be unchanged after every refusal, in 30 configurations: five buffer sizes, three workloads (random, deep shadowing, deep nesting), with the normal hash and with every key forced into one probe cluster. Each run is also compared with a reference model and unwound to empty. Further tests use an allocator that fails after k calls, 5,000-deep shadowing of one name, and scope exit across index rebuilds. All pass under AddressSanitizer and UndefinedBehaviorSanitizer. The argument is informal; nothing was machine-checked.

V. Methodology

Traces. We added logging to TCC (commit `43c7708`) at `sym_find` (lookup), `sym_link` and `global_identifier_push` (declaration of ordinary identifiers), and at every change of its scope level. The patched compiler compiled cJSON 1.7.18, littlefs 2.9.3, zlib 1.3.1, Lua 5.4.7, FreeRTOS-Kernel 11.1.0, Mbed TLS 3.6.2 (three files that TCC rejects are missing), TinyCC itself and the SQLite 3.46.0 amalgamation (Table I). Each translation unit starts with a fresh table. Struct tags, members, labels and macros live in other TCC tables and are not traced.

Tables. STRATA in four configurations; the conventional arena table; METIS-X and its two baselines, unmodified; a hash map per open scope; the undo-log algorithm of LLVM's `ScopedHashTable` [7] on `std::unordered_map` and on a Swiss-table node map [8]; and an interning Swiss flat map with dense per-identifier heads, mirroring the split between interning and resolution in Clang [9]. All implement the same operations with the same semantics.

Measurements. (i) Every result is compared with a reference model built on `std::map`. (ii) Heap is measured through an allocator hook using `malloc_usable_size`; for the two bounded tables we also find by bisection the smallest fixed buffer that replays a trace without a refusal, because growth in steps quantises peak heap. (iii) Time: 31 rounds after two warm-up rounds; in each round every table replays the whole trace once, in an order rotated from round to round. For two tables we take the per-round relative difference and report its median with a 95% bootstrap interval; a difference is *established* when the interval excludes zero. (iv) Per-call latency uses the time-stamp counter with the cost of an empty interval (18 ns) subtracted; it is near the method's resolution

TABLE II
REDUCTION (%) OF STRATA AGAINST METIS-X; NEGATIVE MEANS STRATA IS WORSE. REPLAY: MEDIAN OF PAIRED ROUNDS WITH 95% INTERVAL. LOOKUP, DECLARE, EXIT: P95 LATENCY.

Trace	Heap	Allocs	Replay	95% CI	Lookup	Declare	Exit
cjson	58	99.4	-6	[-21, -1]	-6	47	83
freertos	73	99.0	14	[10, 18]	10	57	77
littlefs	76	99.3	11	[7, 19]	-3	63	88
lua	40	99.2	9	[3, 15]	-26	44	67
mbedtls	38	99.5	19	[16, 21]	28	64	81
sqlite	57	99.9	11	[5, 18]	19	56	70
tinycc	23	99.8	5	[2, 13]	-4	55	72
zlib	67	98.7	11	[4, 15]	0	53	72

and secondary. Runs were pinned to one core of an Intel Core Ultra 7 258V under Linux (WSL 2), GCC 11.4, `-O2`.

VI. RESULTS

Correctness. 96 of 96 table–trace pairs on the compiler traces agree with the reference model.

A. Against METIS-X

Table II: STRATA's peak heap is 23–76% lower (median 58%) with 98.7–99.9% fewer allocator calls. Replay is established faster on 7 of 8 traces and established slower on 1 (cjson); the reduction ranges from -6% to 19%. Declaration p95 is 44–64% lower and scope-exit p95 67–88% lower. Lookup p95 changes by between -26% and 28%: it is not improved.

B. Where the gain comes from

Table III places the conventional arena table between the two. Its smallest fixed buffer is already 55–77% below METIS-X's peak heap, and it replays the traces 11–36% faster than METIS-X. STRATA's buffer is a further 16–18% smaller, 15.8–156.5 KiB or 27–36 bytes per live declaration. But STRATA is established slower than the conventional table on 7 of 8 traces, by 3–63% (median 29%).

At the memory peak the number of live declarations equals the number of distinct live names on all traces but one: C code as TCC compiles it has almost no shadowing there. Interning therefore saves nothing on these traces; STRATA's memory advantage comes from its smaller record and packed names, and its time disadvantage from packing the query and probing an open-addressing index instead of following a short chain that stores full hashes. We did not separate these two causes. On synthetic traces STRATA is established faster than the conventional table on 5 of 9 (those with heavy shadowing, scope churn, a large repeated scope, or very short names) and slower on 3.

C. All tables

Table IV gives replay times. The interning flat map is faster than STRATA on 8 of 8 traces (median 21%); STRATA's peak heap is 44–83% lower than its. The interquartile range of raw replay times has a median of 16% of the median and reaches 39%, and two earlier complete runs of identical code differed by tens of percent on single pairs; this is why we report paired differences only.

TABLE III
METIS-X PEAK HEAP AND THE SMALLEST FIXED BUFFER OF THE TWO BOUNDED TABLES (KIB); REDUCTION (%) OF STRATA AGAINST THE CONVENTIONAL ARENA TABLE IN MEMORY AND IN REPLAY TIME, WITH 95% INTERVAL.

Trace	METIS-X	arena	STRATA	mem.	time	95% CI
cjson	103	35.9	30.0	16	-63	[-66, -54]
freertos	217	53.6	44.0	18	-28	[-37, -24]
littlefs	107	24.2	20.0	18	-35	[-42, -25]
lua	107	43.4	35.9	17	-33	[-39, -23]
mbedtls	234	96.9	80.0	17	-31	[-33, -24]
sqlite	451	169.4	140.4	17	-3	[-7, 6]
tinycc	423	189.8	156.5	18	-23	[-27, -20]
zlib	53	18.9	15.8	17	-14	[-31, -10]

TABLE IV
REPLAY TIME, NS PER TRACE OPERATION, MEDIAN OF 31 ROUNDS (LOWEST IN BOLD)

Trace	STRATA	arena+chains	METIS-X	map/scope	std+undo	Swiss+undo	Swiss int.
cjson	53.4	**33.1**	49.7	60.8	55.0	53.2	39.6
freertos	42.9	**33.1**	48.6	52.6	43.0	46.6	35.1
littlefs	44.2	**32.0**	49.5	69.3	49.5	44.5	36.6
lua	43.5	33.0	47.6	51.2	46.0	46.4	**32.3**
mbedtls	45.5	**35.4**	55.6	51.1	49.6	52.8	38.1
sqlite	29.0	28.9	32.7	60.3	29.4	30.6	**25.0**
tinycc	33.0	26.5	35.0	60.1	33.3	31.5	**26.4**
zlib	33.7	29.3	37.5	49.8	39.8	40.1	**28.7**

D. Options, stress cases, emulated target

Name packing reduces STRATA's minimum buffer by 5.5–9.0%. Limiting the index to half full lowers replay time by a median of 14% and enlarges the buffer by 18–27%. On a synthetic trace that enters a 10,000-declaration scope 20 times STRATA takes 50 ns per operation against 97 for METIS-X. With every key forced into one probe cluster a lookup among 4,096 names takes 1588 ns instead of 14 ns; results stay correct but there is no defence against adversarial keys.

Built for a bare-metal Cortex-M3 and run under QEMU with RAM limited to 64 KiB and allocator stubs that fail, STRATA replayed 5 of the traces from buffers of 16–44 KiB with results identical to the host's, using 792 bytes of stack and 2,557 bytes of code. An emulator shows that the code is correct and fits; it gives no timing.

VII. RELATED WORK

A single hash table with an auxiliary stack unwound at scope exit is described in compiler textbooks [1], [10] and implemented in LLVM's `ScopedHashTable` [7]; chained tables over arenas popped at scope exit are equally standard, and TCC keeps a per-token symbol pointer with symbol stacks [4]. Clang separates an identifier table from a resolver [9]. Fingerprints beside the key follow Swiss tables [11]. Deletion in linear probing normally needs tombstones or shifting [12], [13]; last-in first-out deletion needs neither. Character packing goes back to RADIX-50 [5], stack-like allocation by lifetime to arena allocators [14]. Our search of prior work was not systematic.

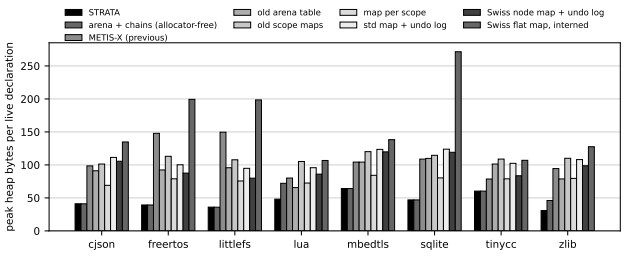


Fig. 2. Peak heap per live declaration on the compiler traces.

VIII. Limitations

Neither bounded table was integrated into a compiler; replaying TCC's calls measures tables, not compilation. The target run is an emulator, not a device. Traces reflect one compiler and C only. Timings come from one laptop under a hypervisor. All baselines except METIS-X's were written by us after known designs; they are not the production code of LLVM or Clang. The exception-safety defect of METIS-X was found by inspection. The correctness argument is informal.

IX. Conclusion

On compiler-generated traces, most of what a bounded-memory symbol table gains over METIS-X comes from keeping names and entries in one allocation-free arena, which a conventional chained table already does and does faster. Interning with an undo stack and packed names saves a further sixth of the memory, gives lookups that do not depend on shadowing depth and wins when shadowing is heavy, at the price of slower operation on ordinary C code. Which of the two is preferable depends on whether memory or time is the binding constraint. The next steps are to replace TCC's own tables with each design and measure compilation, and to run on a device.

Acknowledgment

The audit, implementation, experiments and this draft were produced with an AI coding assistant (Claude, Anthropic) working in the first author's environment. The authors are responsible for verifying the code and the claims before any submission.

Data and Code Availability

Source code, traces, raw measurements (run `20261011-165103-v3`) and the scripts that generate every table and figure are in the project repository. METIS-X and the Swiss-table library are fetched at pinned versions and are not redistributed.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[2] A. Jain and S. Singh, "METIS: symbol-table research repository, commit 7b3dacf," https://github.com/adhyan-jain/Metis, 2026.

[3] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, 1985.

[4] F. Bellard and TinyCC contributors, "Tiny C Compiler," https://repo.or.cz/tinycc.git, commit 43c7708.

[5] Wikipedia contributors, "DEC RADIX 50," https://en.wikipedia.org/wiki/DEC_RADIX_50.

[6] A. Appleby, "MurmurHash3, SMHasher," https://github.com/aappleby/smhasher.

[7] LLVM Project, "ScopedHashTable.h, LLVM ADT library," https://llvm.org/doxygen/ScopedHashTable_8h_source.html.

[8] G. Popovitch, "The parallel hashmap, version 1.4.1," https://github.com/greg7mdp/parallel-hashmap.

[9] LLVM Project, "IdentifierTable.cpp and IdentifierResolver.h, Clang," https://github.com/llvm/llvm-project/tree/main/clang.

[10] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

[11] Abseil, "Swiss tables design notes," https://abseil.io/about/design/swisstables.

[12] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[13] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[14] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.